

Profiling and Pruning Visual-Token KV Caches in Qwen2.5-VL

Does Mustafar-style unstructured magnitude pruning, developed for text-only caches, transfer to visual tokens?

Shaashvat Shetty
CASL, University of Maryland

Summer–Fall 2026

Abstract

Vision-language models turn each image into hundreds or thousands of visual tokens. Every one of those tokens adds a Key and a Value entry to the KV cache at every layer, so the cache quickly becomes the main memory cost of running the model. Pruning methods like Mustafar shrink text-only KV caches by dropping small-magnitude elements, but the structure they rely on — a few large elements and outlier channels carrying most of the signal — has only ever been measured in text-token caches. This project asks whether visual-token caches in Qwen2.5-VL-3B-Instruct have that same structure, and whether Mustafar’s pruning rule works on them. Profiling shows that visual and text Key caches share the same layer-wise concentration pattern at similar magnitudes. Pruning experiments back this up: magnitude pruning of visual Keys holds accuracy up to 90% sparsity on TextVQA and DocVQA, while random pruning at the same sparsity fails immediately. A Key 70% / Value 80% setting cuts measured stored-cache memory by about 55–57% at under a two-point accuracy cost, using a real bitmap-compressed cache in the style of Coruscant and Mustafar, with measured savings landing within one to two points of the calculated ones. Mustafar’s CUDA kernel, which reads that compressed form directly, is verified correct here but runs slower than PyTorch’s FlashAttention-2 decode path at this model’s attention geometry — an honest negative result and the focus of ongoing work.

1 The Problem

Vision-language models (VLMs) extend a language model so it can read images. The cost is token count: a single image becomes hundreds to several thousand visual tokens before the language model ever sees it. Each of those tokens produces a Key and a Value cache entry at every transformer layer, and those entries stay in GPU memory for the whole generation. On a single DocVQA page, Qwen2.5-VL produces over a thousand visual tokens. At that scale the KV cache — not the model’s weights — becomes the main cost of serving the model, in memory, in memory bandwidth, and in decoding latency.

Recent work shrinks the KV cache of *text-only* language models by using structure inside the cache itself: a small number of large-magnitude elements and outlier channels carry most of the signal, so the rest can be pruned with little quality loss. Mustafar turns this into a simple rule — per-token magnitude pruning at the element level. But that structure was only ever measured in text-token caches. No one had checked whether a visual token’s Key or Value vector, which comes from a vision encoder rather than from language modeling, has the same kind of magnitude

concentration. If it does, text-only pruning rules transfer directly. If it does not, applying the same rule to visual tokens could quietly throw away visual information the model needs.

This project answers that question by measuring first: **do visual-token KV caches have the same magnitude and outlier-channel structure as text-token caches, and does Mustafar-style pruning behave the same way on them?** Profiling comes first; pruning is the validation step, not the assumed conclusion.

2 Related Work

Prior work falls into three groups, and the gap between them is exactly this project’s question.

Text-only KV-cache compression. Mustafar removes small-magnitude Key/Value elements while keeping large elements and outlier channels, and stores the result in a compressed bitmap form so the savings are real. KIVI shows Key and Value caches have different distributions and quantizes them differently — per-channel for Keys, per-token for Values. Both study fine-grained, element-level structure. Neither touches a visual token.

VLM cache and token compression. VL-Cache and AirCache show that visual and text tokens should not be compressed the same way, setting cache budgets by layer and by how relevant visual tokens are to the prompt. SparseVLM, FastVLM, and Focus show that many visual tokens are redundant and can be dropped or merged. All five work at the token, layer, or vision-encoder level. None looks at the individual elements or channels *inside* a visual token’s Key/Value vector — the level where Mustafar and KIVI operate.

Bitmap formats and kernels for unstructured sparsity. Coruscant, from the same group as Mustafar, is the systems half of this story. Unstructured sparsity is normally hard to use on a GPU: the surviving values sit at irregular positions, and finding them costs more than the skipped work saves. Coruscant’s answer is a bitmap format. Data is cut into small tiles; each tile keeps only its nonzero values, packed together, plus one bit per element marking which positions survived. A matching GPU kernel then works on this compressed form *directly* — it uses the bitmap to locate each surviving value and never rebuilds the dense matrix. Coruscant applies this to model weights, which are fixed after training. Mustafar carries the same format and kernel design to the KV cache, which is created on the fly during inference. This project implements that format for visual-token caches (Section 3) and runs Mustafar’s kernel on it during decode (Section 4.5). One more baseline note: the dense attention this paper compares against is PyTorch’s bundled FlashAttention-2 decode path, confirmed by profiling rather than assumed — so the kernel comparison later is against a real, modern baseline.

Together, these works leave one gap: no one had checked whether visual-token KV caches have the text-like element-level structure that all of the above depends on. This project fills that gap, then runs the same kind of validation Mustafar ran — a full pruning and evaluation study — on the modality it was never tested on.

3 Approach and Implementation

All experiments use **Qwen2.5-VL-3B-Instruct**. It marks image placeholder tokens explicitly in the input sequence, which makes the visual/text split verifiable rather than guessed.

Capture and profiling. Prefill Key/Value tensors are saved per layer and split into visual-token and text-token positions. The split is then profiled: magnitude distributions, channel concentration by layer and by head, and visual-to-text magnitude ratios, on calibration sets of up to 500 images

(COCO, ImageNet, TextVQA). This step answers the structural question directly and is reported first in Section 4.

Live pruning during generation. Guided by the profiling result, Mustafar’s token-wise magnitude rule is applied during real generation — not just scored offline — to visual-token cache entries only, on Keys, Values, or both. Text-token entries stay dense throughout, so any accuracy change can be blamed on visual-token pruning alone. Accuracy scoring uses **dense-zero** semantics: pruned entries are set to zero but tensors stay dense, exactly as Mustafar’s own accuracy evaluation does. Memory is measured on a separate, real storage path described below.

Controls. Three controls isolate what actually drives any accuracy change:

- **Random** — same achieved sparsity, entries chosen at random.
- **Uniform** — the magnitude rule applied to text Keys as well as visual Keys.
- **Structured** — whole channels dropped by the same magnitude score, at matched sparsity.

Every compared run is **pixel-matched** (a fixed visual-token budget), so visual-token counts are identical across arms; a pairing script checks this per example and found zero mismatches.

Compressed storage. The compression technique follows the bitmap format that Coruscant introduced for weights and Mustafar adapted for KV caches, implemented here for visual-token cache entries. A pruned Key or Value tensor is cut into 64-element tiles. Each tile stores three things: its surviving values packed tightly together, a 64-bit bitmap with one bit per element saying which positions survived, and a 32-bit offset saying where that tile’s values start in the packed array. Nonzero counts are padded to multiples of eight so memory reads stay aligned. During generation, visual Key and Value entries are stored in this form, and the cache reports its *actual byte count* — so the memory results in Section 4.4 are measured, not calculated. A CPU test suite confirms the compressed path produces bit-identical outputs to the dense-zero path, and a full parity run on real examples passed with identical predictions.

The kernel. The point of the bitmap format is that attention never has to decompress it. Mustafar’s CUDA kernel reads the packed values and the bitmap directly: for each tile, the bitmap tells it which elements exist, and it multiplies only those. Without such a kernel, the cache would have to be unpacked back to dense form at every decode step, and the memory saving would vanish exactly when the cache is being read. The kernel was vendored unmodified from Mustafar’s repository, integrated through its own reference calling convention, and verified for correctness before any speed measurement (Section 4.5).

Evaluation. Accuracy uses fixed-reference, right/wrong metrics rather than text-similarity scores: **TextVQA** (relaxed VQA accuracy) and **DocVQA** (ANLS), 500 examples each. MMMU was in the original plan but was dropped: its short multiple-choice answers are already decided during dense prefill, so decode-time pruning never gets a chance to show up in the score.

4 Results

4.1 Visual Key caches share text’s magnitude structure

The profiling question comes first. Across all 36 layers, visual and text Key caches trace the same shape: both start highly concentrated in a few channels, settle into a stable middle band, and spike together at the same layers — most sharply near layer 27, where both jump from about 0.15 to over 0.30 in lockstep. Value caches for both modalities stay lower and flatter than Keys. This is the same pattern Mustafar and KIVI report for text-only models: Key caches concentrate into outlier channels, Value caches do not.

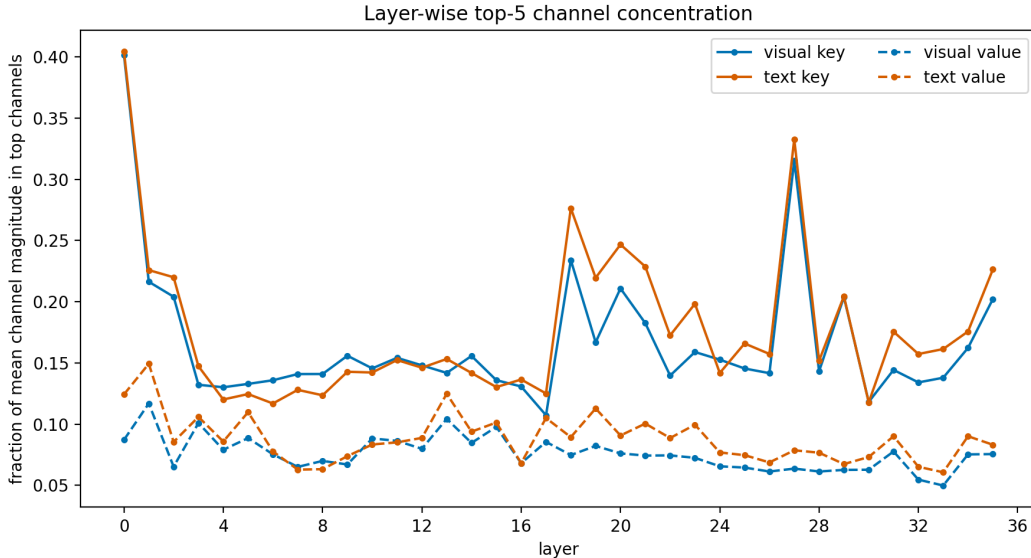


Figure 1: Fraction of mean channel magnitude carried by the top-5 channels, by layer (COCO calibration, $n = 500$). Visual Key (solid blue) and text Key (solid orange) rise and fall together across all 36 layers, including the shared spike near layer 27. Visual and text Value curves (dashed) stay lower and flatter. Keys concentrate into outlier channels just as text-only work reports — and the pattern does not depend on modality.

The size gap between modalities is also small. Comparing mean absolute Key magnitude, visual and text tokens stay within roughly a factor of two of each other at nearly every layer and head — a $\log_2$ ratio mostly between -0.3 and $+0.3$ — not orders of magnitude apart.

Visual-token Key caches carry the same layer-wise outlier-channel concentration as text-token Key caches, at comparable magnitude. There is no separate “visual regime” for Mustafar’s rule to miss — the premise behind text-only magnitude pruning holds for visual tokens too.

4.2 The pruning behavior confirms it

If that structure is real, magnitude selection should behave the way it does for text: nearly lossless at high sparsity, while a control that ignores magnitude should fail. That is exactly what happens.

Two things follow from Figure 3. First, **selecting by magnitude is what matters**: the random control’s immediate collapse shows the visual Key cache really does contain exploitable

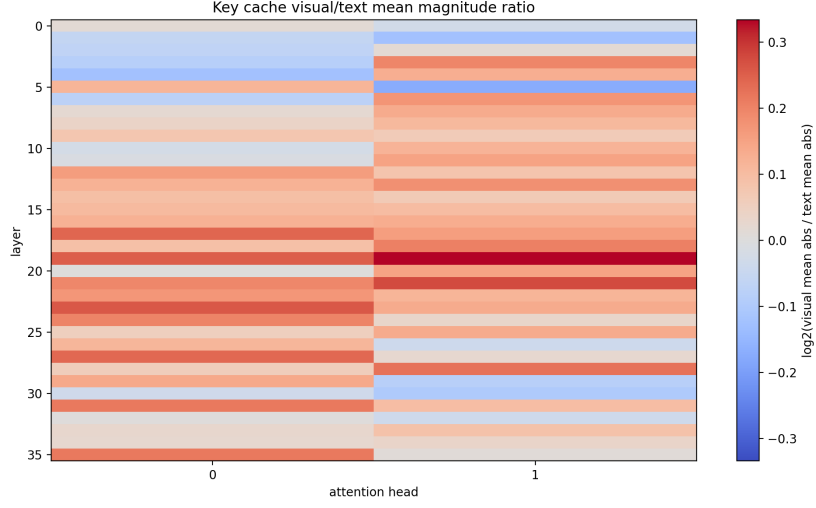


Figure 2: $\log_2(\text{visual mean } |\text{Key}| / \text{text mean } |\text{Key}|)$ by layer and attention head. Values cluster near zero, meaning visual and text magnitudes stay within about $2\times$ of each other almost everywhere. The darkest cell — the largest visual-leaning gap — sits in the same layer 19–20 band as the concentration spike in Figure 1.

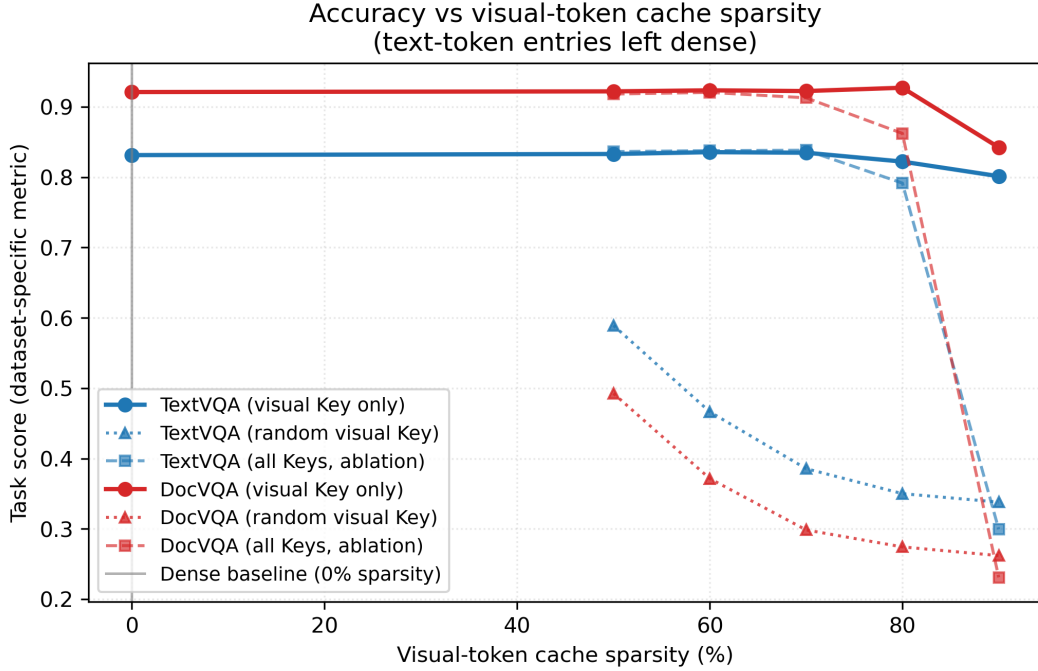


Figure 3: Task accuracy vs. visual-token Key sparsity, text entries left dense ($n = 500$ per dataset per point). **Magnitude** (solid) is near-flat to 90% sparsity on both datasets. **Random** selection at the identical achieved sparsity (dotted) collapses immediately — DocVQA falls from 0.92 to 0.49 at just 50% sparsity. **Uniform** (dashed, text Keys pruned too) tracks magnitude closely through 70%, then falls off a cliff between 80% and 90%.

structure, and Mustafar’s rule is finding it. Second, the modality difference is narrower than “visual tokens are special” — **text Keys are also safely magnitude-prunable to 70%**. The real danger zone is 80–90% sparsity in general, not text tokens in particular. Visual-only pruning stays safe at 90% because leaving text dense limits how much of the sequence is under that much pressure.

4.3 Key, Value, and both together

Pruning Keys or Values alone is safe to 90% sparsity on both datasets. Pruning both together is safe only to 70%, and past that it fails harder than either alone: at 90% sparsity, the Key-only and Value-only arms beat the combined arm by 0.15–0.25 accuracy points, with 95% confidence intervals that exclude zero on both datasets. A grid search over separate Key and Value sparsities finds **Key 70% / Value 80%** to be the best operating point on both benchmarks (Table 1).

Dataset	Dense baseline	Key 70% / Value 80%	Score drop	KV reduction (calc.)
TextVQA (relaxed accuracy)	0.830	0.817	−0.013	57.7%
DocVQA (ANLS)	0.921	0.915	−0.006	58.1%

Table 1: Best asymmetric Key/Value sparsity operating point, 500 examples per dataset. The KV reduction column is the calculated bitmap footprint; Table 2 checks it against measured bytes.

4.4 Memory: measured vs. calculated

Memory savings in this project come in two forms, and it is worth being clear about the difference. The *calculated* number comes from an analytic model of the bitmap format: given a sparsity level, it computes how many bytes the tiles, bitmaps, and offsets should take. The *measured* number is the actual byte count of the compressed cache while the model is generating. The evaluation plots (for example, the memory bar in Figure 4) use the calculated footprint, because the accuracy sweeps run on the dense-zero path; the compressed-cache and kernel runs then check that calculation against reality.

	Calculated	Measured (compressed cache)	Measured (kernel path)
TextVQA	57.7%	56.5%	55.2%
DocVQA	58.0%	56.7%	56.8%

Table 2: Stored cache-object memory reduction at Key 70% / Value 80%, visual-only with text left dense. Measured values are real byte counts recorded during generation.

The measured savings land within one to two percentage points of the calculated ones, on both datasets and on both storage paths. The measured number is slightly smaller because the real format pays small costs the calculation treats ideally — the per-tile bitmaps and offsets, and the padding of nonzero counts to multiples of eight. The agreement is the point: the memory claim rests on a format that actually runs, not on a formula. These numbers are always *stored cache-object memory* — the cache tensors themselves — not total GPU memory.

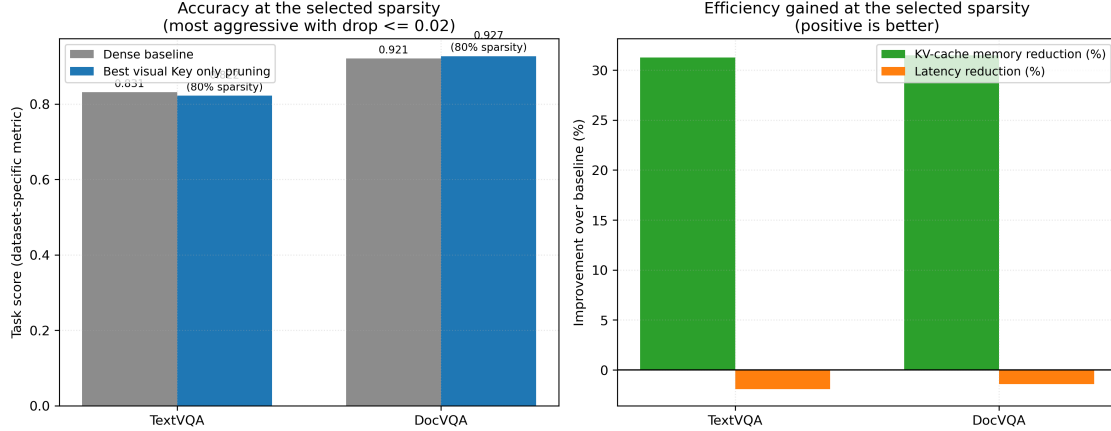


Figure 4: Left: accuracy at the most aggressive sparsity holding score drop ≤ 0.02 (Key-only sweep, 80% sparsity). Right: the same operating point’s memory and latency change. The memory bar is the calculated bitmap footprint (about 30% for a Key-only sweep, since Values stay dense); Table 2 verifies the calculation against measured bytes. **Latency does not improve** — expected, because the accuracy path zeroes entries in dense tensors rather than compacting them. Real speedups have to come from the kernel, discussed next.

4.5 The kernel: correct, but slower than dense

The kernel exists to make the memory saving free at decode time. Attention has to read the whole cache at every generated token; if the cache had to be decompressed before each read, the saving would disappear right when it matters. Mustafar’s CUDA kernel avoids this by computing attention straight from the compressed tiles — the bitmap tells it which elements exist, and it multiplies only those, never unpacking to a dense tensor.

Correctness checks out. Integrated through its upstream reference calling convention and compared against the dense-zero path, the kernel produced **zero prediction mismatches across 500 examples on both TextVQA and DocVQA**. Speed does not check out, and Figure 5 shows why in two parts.

The comparison is fair: profiling confirmed the dense baseline dispatches to `flash_fwd_splitkv_kernel`, PyTorch’s bundled FlashAttention-2 decode path — the same class of kernel Mustafar’s own paper compares against. The result does not contradict Mustafar’s published numbers, which are end-to-end tokens per second on Llama models with wider attention geometries. It says the speed advantage does not carry over to this VLM’s narrow 16-query/2-KV-head geometry, at least not without more kernel work.

One separate finding concerns the compression step itself. The Triton converters that pack the cache into bitmap form are cheap once amortized — 0.04–0.17% of a decode step, spread over 1,024 generated tokens, charged the same way Mustafar’s paper charges it. But they issue 10 to 66 device-to-host memory copies per call. Those round-trips through the CPU are a plausible mechanism for the time-to-first-token increase Mustafar’s own paper reports, independent of anything in this project’s integration.

Mustafar CUDA kernel is slower than dense FlashAttention at this model's 16Q/2KV geometry

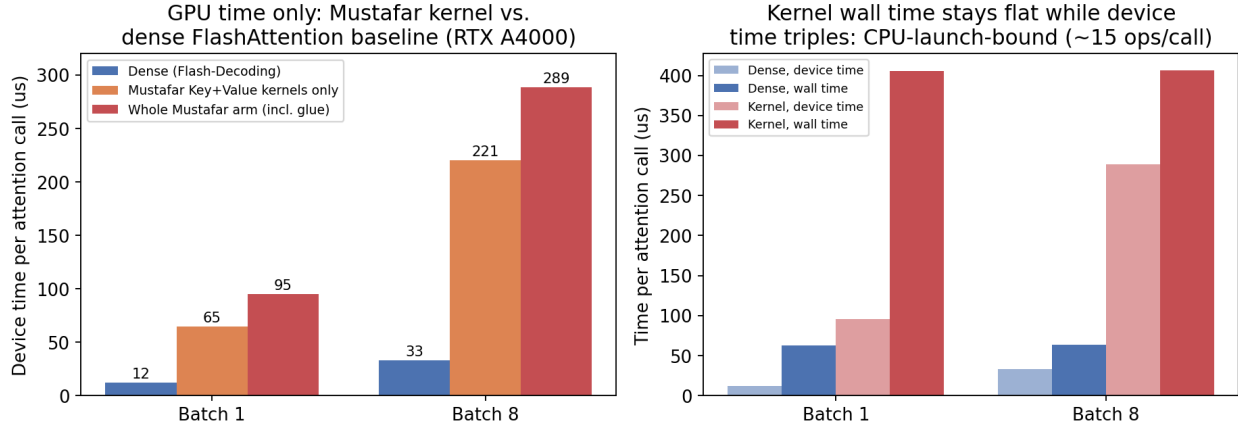


Figure 5: RTX A4000, Qwen2.5-VL’s 16-query/2-KV-head geometry, Key 70%/Value 80%. **Left:** counting GPU time alone, with all wrapper code excluded, Mustafar’s Key and Value kernels are **5.3× slower at batch 1** and **6.7× slower at batch 8** than PyTorch’s Flash-Decoding baseline. **Right:** wall time on the kernel path stays flat near 405µs from batch 1 to batch 8 even as GPU work triples — the path issues about 15 separate CUDA launches per call versus dense’s 2, so it is limited by CPU launch overhead, not GPU throughput.

5 Implications

Practical. A VLM serving system can cut stored KV-cache memory roughly in half at under a two-point accuracy cost, by pruning visual-token entries almost exclusively. Because this works inside each token’s cache vector rather than removing whole tokens, it composes with token-dropping and quantization methods instead of competing with them.

Scientific. Text-only KV-cache compression assumptions *do* transfer to visual tokens in this model: the same magnitude structure is there, and the same rule exploits it the same way. The modality difference that remains is a difference in sparsity *tolerance* at the extreme end (80–90%), not a difference in which rule applies.

Systems. The memory win is real, and the measured-versus-calculated agreement in Table 2 shows the bitmap format delivers what its arithmetic promises. The speed win is a different story: it depends heavily on attention-head geometry and kernel engineering, and it did not materialize here against a modern FlashAttention baseline. That negative result is stated plainly rather than worked around.

Limitations. One model (Qwen2.5-VL-3B); accuracy scored on the dense-zero path, where pruned entries are zeroed rather than physically removed; the first generated token always comes from dense prefill, and VQA answers are short, so the decode-time test of pruning is conservative.

6 Future Goals

Importance-aware, per-token pruning. Right now every visual token gets the same sparsity target. Section 4.2 suggests this leaves something on the table: selection quality, not modality, is the main driver of accuracy retention, and the random control shows the cache’s usable structure is spread unevenly across tokens. Some image patches surely matter less than others for a given

query — a question about text in a document’s corner makes most of the page’s white space irrelevant. The current work in progress tests whether that per-token importance can be estimated cheaply and used to set a *non-uniform* budget: prune an unimportant patch at, say, 90% while an important one stays near 70%, instead of one fixed rate for all. The open question is whether such an importance signal can be computed cheaply enough at inference time, and whether it beats the fixed-sparsity curve in Figure 3 at the same average memory budget.

Closing the kernel speed gap. Figure 5 is the clearest unsolved problem in this project. The memory result stands on its own; the speed result is currently negative, for two concrete reasons:

GPU time. Mustafar’s kernel is $5\text{--}7\times$ slower than Flash-Decoding at this model’s 16Q/2KV geometry even before counting integration overhead — likely because per-query-head batching re-reads each compressed KV head eight times and leaves most tensor-core rows idle.

Launch overhead. On top of that, the path issues about 15 separate CPU-side kernel launches per attention call versus dense’s 2, which is why wall time sits flat near $405\mu\text{s}$ across batch sizes 1–8 while GPU work triples underneath. Fusing that glue would bring wall time down toward the GPU-time floor — but that floor is still several times slower than dense, so fusion alone will not close the gap.

There is also a more basic question to answer before more kernel tuning. At batch 1, this model’s dense KV cache is only about 1 MB per layer — small enough that it may stay resident in the GPU’s L2 cache. Mustafar’s whole performance case assumes decoding is limited by DRAM bandwidth; if the working set never leaves L2 at this context length, that assumption is not actually being tested, and the comparison in Figure 5 would need to move to longer contexts or larger batches to be a fair test of the kernel’s design point. Getting a real end-to-end, batched `model.generate` timing — not just per-layer numbers — is the next concrete step, and requires extending the kernel’s decode path past its current batch-1 limit.

Continuing into the school year. This project continues as an independent study with the supervising professor through the coming school year, with the two threads above — importance-aware visual-token pruning and closing the kernel’s speed gap — as its main tracks.

References

- [1] D. Joo, H. Hosseini, R. Hadidi, and B. Asgari, “MUSTAFAR: Promoting unstructured sparsity for KV cache pruning in LLM inference,” *Advances in Neural Information Processing Systems*, vol. 39, 2025.
- [2] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, “KIVI: A tuning-free asymmetric 2-bit quantization for KV cache,” *ICML*, PMLR vol. 235, 2024, pp. 32332–32344.
- [3] D. Tu, D. Vashchilenko, Y. Lu, and P. Xu, “VL-Cache: Sparsity and modality-aware KV cache compression for vision-language model inference acceleration,” *ICLR*, 2025.
- [4] K. Huang, H. Zou, B. Wang, Y. Xi, Z. Xie, and H. Wang, “AirCache: Activating inter-modal relevancy KV cache compression for efficient large vision-language model inference,” *arXiv:2503.23956*, 2025.

- [5] Y. Zhang, C.-K. Fan, J. Ma, W. Zheng, T. Huang, K. Cheng, D. A. Gudovskiy, T. Okuno, Y. Nakata, K. Keutzer, and S. Zhang, “SparseVLM: Visual token sparsification for efficient vision-language model inference,” *ICML*, PMLR vol. 267, 2025, pp. 74840–74857.
- [6] P. K. A. Vasu et al., “FastVLM: Efficient vision encoding for vision language models,” *CVPR*, 2025, pp. 19769–19780.
- [7] C. Wei et al., “Focus: A streaming concentration architecture for efficient vision-language models,” *IEEE HPCA*, 2026, pp. 1–18.
- [8] D. Joo, H. Hosseini, R. Hadidi, and B. Asgari, “Coruscant: Co-designing GPU kernel and sparse tensor core to advocate unstructured sparsity in efficient LLM inference,” *MICRO*, 2025, pp. 232–245.